

Formal Verification of InfluenceDiagrams.jl

Simon Frost

2026-09-07

Contents

InfluenceDiagramsProofs	1
What is formalised	1
Correspondence with SPEC and the Julia API	2
InfluenceDiagramsProofs.Finite.InfluenceDiagram	3
Generators and validity	4
Policies and strategies	5
InfluenceDiagramsProofs.Finite.Instantiate	6
InfluenceDiagramsProofs.Finite.ExpectedUtility	8
Utilities	8
Fixing a decision	9
InfluenceDiagramsProofs.Finite.Information	10
Roadmap (contains sorry)	11
Proposition 7 — DVE correctness (SPEC §33, §61)	11

InfluenceDiagramsProofs

```
import BayesianNetworksProofs.Schema.BayesNet
```

Lean 4 / Mathlib formalisation accompanying InfluenceDiagrams.jl (toolchain leanprover/lean4:v4.30.0, Mathlib tag v4.30.0; ADR 0005). This document is generated from the Lean sources by [mdgen](#): the prose is the module docstrings and the code blocks are the verbatim, machine-checked sources. Every declaration outside the final “Roadmap” section is built by `lake build --wfail` and its axioms are printed by `Audit.lean` (only `propext`, `Classical.choice`, `Quot.sound`).

The project depends by path on `BayesianNetworks.jl/proofs` (Lake package `bayesian_networks_proofs`), which supplies the finite Bayesian-network model (`FinBayesNet`, `kernels`, `Local`, `Normalised`, `Closed`, `TopoOrder`, `joint`, Propositions 1 and 4) and the ACSet schema terms; the influence-diagram schema `schInfluenceDiagram` is defined there and re-emitted here by `lake exe emit_schema`.

What is formalised

`InfluenceDiagrams.jl` represents an influence diagram as an ACSet on `SchInfluenceDiagram` (SPEC §24: decisions, information arcs, utilities, decision precedence). A complete strategy turns it into a Bayesian network (`instantiate`, SPEC §28) whose joint gives the expected utility (`expected_utility`, SPEC §29–§31); fixing a decision (`fix_decision`, SPEC §41) is an intervention, and adding information arcs cannot lower the optimal expected utility (SPEC §34–§35, §55.7). All results are over an arbitrary commutative semiring R .

Part	Module	Content
1	Finite/InfluenceDiagram.lean	FinInfluenceDiagram extends FinBayesNet (decisions, action, info, utility nodes, uscope); validity (GeneratorsDisjoint, GeneratorsCover, Closed, closed_iff, IDOrder); Policy, Policy.ofFun, Policy.const, Strategy, Strategy.fix.
2	Finite/Instantiate.lean	Proposition 5: instantiate, strategyKernel, closed_instantiate, IDOrder.toTopoOrder, local_instantiate, normalised_instantiate, sum_joint_instantiate_eq_one, joint_instantiate.
3	Finite/ExpectedUtility.lean	Proposition 6: Utility, totalUtility, expectedUtility, expectedUtility_eq, linearity, expectedUtility_eq_sum, expectedUtility_const; fix_decision as a hard intervention (strategyKernel_fix_eq_intervene, joint_fix, expectedUtility_fix).
4	Finite/Information.lean	Information monotonicity: localOn_mono, withInfo, Policy.enlarge, Strategy.enlarge, expectedUtility_enlarge, exists_strategy_enlarged_eq.
—	Roadmap.lean	Proposition 7 (single-decision shadow, sorry); not in the default target.

Correspondence with SPEC and the Julia API

SPEC	Lean	Julia (InfluenceDiagrams.jl)
§26–§27 policies, strategies	Policy, Policy.ofFun, Strategy	DeterministicPolicy, StochasticPolicy, Strategy instantiate(id, strategy) → BayesNet (validate(closed=true) succeeds)
§28, §61 Prop 5 — policy instantiation yields a valid BN	instantiate, strategyKernel, closed_instantiate, IDOrder.toTopoOrder, sum_joint_instantiate_eq_one, joint_instantiate	expected_utility(id, strategy) (instantiate, enumerate the joint, sum utilities, take the expectation)
§29–§31, §61 Prop 6 — $EU(\sigma) = E_{P_\sigma}[U]$	Utility, totalUtility, expectedUtility, expectedUtility_eq, expectedUtility_eq_sum	fix_decision(model, :D ⇒ :a) = do_intervention on the instantiated network
§41 fixed decision	Policy.const, Strategy.fix, strategyKernel_fix_eq_intervene, joint_fix	expected_value_of_information is non-negative
§34–§35, §55.7 information monotonicity	withInfo, Strategy.enlarge, exists_strategy_enlarged_eq	optimize(id; backend=Exhaustive()) versus DVE, property tests (SPEC §55.6)
§33, §61 Prop 7 — DVE correctness	Roadmap.exists_deterministic_optimal (single-decision shadow, unproved)	SchInfluenceDiagram, compared with schemas/influence_diagram.schema.json
§24 schema	schInfluenceDiagram (BayesianNetworks project), lake emit_schema	

```
namespace InfluenceDiagramsProofs
```

```
-- Smoke lemma so that the axiom audit always has a first line. -/
theorem smoke : (1 : Nat) + 1 = 2 := rfl
```

```
end InfluenceDiagramsProofs
```

InfluenceDiagramsProofs.Finite.InfluenceDiagram

```
import BayesianNetworksProofs.Finite.BayesNet
import Mathlib.Algebra.Ring.Defs
import Mathlib.Algebra.BigOperators.Group.Finset.Piecewise
```

Concrete finite influence diagrams — the semantic model behind `InfluenceDiagrams.jl` (SPEC §23–§27, §36, §37).

A `FinInfluenceDiagram` extends the finite Bayesian-network shape `FinBayesNet` of the `BayesianNetworks` proofs project (chance variables V , chance mechanisms M , states, target, parents) with

- a finite type D of decisions, each controlling an action variable and reading an information set `info_d` (the `Decision / InformationInput` rows of `SchInfluenceDiagram`);
- a finite type U of utility nodes, each reading a scope `uscope_j` (the `Utility / UtilityInput` rows).

Kernels (chance), policies (decisions) and utilities live outside the structure, exactly as `kernel_ref / utility_ref` point outside the `ACSet` in Julia.

Validity (SPEC §37):

- `GeneratorsDisjoint` — no variable is both a mechanism target and an action (item 2);
- `GeneratorsCover` — every variable is a mechanism target or an action;
- `Closed` — $\text{Sum.elim target action} : M \otimes D \rightarrow V$ is bijective, i.e. every variable has exactly one generator, mechanism or decision (items 1, 2 together with `closed_iff`);
- `IDOrder` — a topological order of the variables in which parents precede mechanism targets and information variables precede actions (items 4, 8, 11).

A `Policy id R d` is a stochastic kernel $\text{Assignment} \rightarrow \text{states}(\text{action } d) \rightarrow R$ that reads only `info_d` (`LocalOn`) and is normalised (SPEC §26); a deterministic policy is `Policy.ofFun`, a constant one `Policy.const`. A `Strategy id R` is one policy per decision — complete by construction (SPEC §27).

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

/-- A finite influence-diagram *shape*: a finite Bayesian-network shape plus decisions
(action variable, information set) and utility nodes (scope). -/
structure FinInfluenceDiagram extends FinBayesNet where
  /-- Decisions (one per 'Decision' row of the ACSet). -/
  D : Type
  [fintypeD : Fintype D]
  [decD : DecidableEq D]
  /-- The action variable controlled by each decision ('decision_variable'). -/
  action : D → V
  /-- The information set 'I_D' of each decision (the 'InformationInput' rows). -/
  info : D → Finset V
  /-- Utility nodes (one per 'Utility' row). -/
  U : Type
  [fintypeU : Fintype U]
  [decU : DecidableEq U]
  /-- The variables read by each utility node (the 'UtilityInput' rows). -/
  uscope : U → Finset V

attribute [instance] FinInfluenceDiagram.fintypeD FinInfluenceDiagram.decD
  FinInfluenceDiagram.fintypeU FinInfluenceDiagram.decU

namespace FinInfluenceDiagram

variable (id : FinInfluenceDiagram)
```

Generators and validity

```

/-- Every variable is generated by a mechanism or by a decision. -/
def generator : id.M ⊗ id.D → id.V := Sum.elim id.target id.action

@[simp] theorem generator_inl (m : id.M) : id.generator (.inl m) = id.target m := rfl

@[simp] theorem generator_inr (d : id.D) : id.generator (.inr d) = id.action d := rfl

/-- No variable is both a mechanism target and a decision action (SPEC §37 item 2: an
uninstantiated action has no chance mechanism). -/
def GeneratorsDisjoint : Prop := ∀ m d, id.target m ≠ id.action d

/-- Every variable is a mechanism target or a decision action. -/
def GeneratorsCover : Prop := ∀ v, (∃ m, id.target m = v) ∨ (∃ d, id.action d = v)

/-- Closed influence diagram: every variable has exactly one generator – a chance mechanism or a
decision ('validate' items 1 and 2 plus closedness of the chance part). -/
def Closed : Prop := Function.Bijective id.generator

/-- 'Closed' unpacked: both generator maps are injective, disjoint and jointly cover 'V'. -/
theorem closed_iff :
  id.Closed ↔ Function.Injective id.target ∧ Function.Injective id.action ∧
  id.GeneratorsDisjoint ∧ id.GeneratorsCover := by
  constructor
  · rintro ⟨hinj, hsurj⟩
    refine ⟨fun m m' h ⇒ ?_, fun d d' h ⇒ ?_, fun m d h ⇒ ?_, fun v ⇒ ?_⟩
    · exact Sum.inl_injective (hinj (a₁ := .inl m) (a₂ := .inl m') h)
    · exact Sum.inr_injective (hinj (a₁ := .inr d) (a₂ := .inr d') h)
    · exact Sum.inl_ne_inr (hinj (a₁ := .inl m) (a₂ := .inr d) h)
    · obtain ⟨g, hg⟩ := hsurj v
      cases g with
      | inl m ⇒ exact Or.inl ⟨m, hg⟩
      | inr d ⇒ exact Or.inr ⟨d, hg⟩
  · rintro ⟨hT, hA, hdisj, hcover⟩
    refine ⟨fun g g' h ⇒ ?_, fun v ⇒ ?_⟩
    · cases g with
      | inl m ⇒
        cases g' with
        | inl m' ⇒ exact congrArg Sum.inl (hT h)
        | inr d' ⇒ exact absurd h (hdisj m d')
      | inr d ⇒
        cases g' with
        | inl m' ⇒ exact absurd h.symm (hdisj m' d)
        | inr d' ⇒ exact congrArg Sum.inr (hA h)
    · rcases hcover v with ⟨m, hm⟩ | ⟨d, hd⟩
      · exact ⟨.inl m, hm⟩
      · exact ⟨.inr d, hd⟩

/-- Build 'Closed' from its four components. -/
theorem Closed.of (hT : Function.Injective id.target) (hA : Function.Injective id.action)
  (hdisj : id.GeneratorsDisjoint) (hcover : id.GeneratorsCover) : id.Closed :=
  id.closed_iff.2 ⟨hT, hA, hdisj, hcover⟩

/-- A topological order of the influence diagram: a duplicate-free complete list of the variables
in which every parent of a mechanism precedes its target and every information variable of a
decision precedes its action; no generator reads the variable it generates (SPEC §37 items 4,
8, 11; the decision precedence is the induced order of the actions). -/
structure IDOrder where
  order : List id.V
  nodup : order.Nodup
  complete : ∀ v, v ∈ order
  /-- For 'a' before 'b', 'b' is not a parent of the mechanism generating 'a'. -/
  parents_before : order.Pairwise (fun a b ⇒ ∀ m, id.target m = a → b ∉ id.parents m)
  /-- For 'a' before 'b', 'b' is not an information variable of the decision controlling 'a'
  (information is available before the decision is taken). -/

```

```

info_before_action : order.Pairwise (fun a b  $\Rightarrow$   $\forall$  d, id.action d = a  $\rightarrow$  b  $\notin$  id.info d)
no_self :  $\forall$  m, id.target m  $\notin$  id.parents m
no_self_info :  $\forall$  d, id.action d  $\notin$  id.info d

```

Policies and strategies

```

variable {id} {R : Type} [CommSemiring R]

/-- A (stochastic) policy for decision 'd': a normalised kernel from assignments to actions that
reads only the information set 'info d' (SPEC §26, §37 item 9). -/
structure Policy (id : FinInfluenceDiagram) (R : Type) [CommSemiring R] (d : id.D) where
  /- 'kernel x a' is the probability of choosing action 'a' given the assignment 'x'. -/
  kernel : id.Assignment  $\rightarrow$  id.states (id.action d)  $\rightarrow$  R
  /- The policy reads only its information set. -/
  localOn : LocalOn (id.info d) kernel
  /- The policy is a stochastic map. -/
  normalised :  $\forall$  x,  $\sum$  a, kernel x a = 1

/-- A complete strategy: one policy per decision (SPEC §27, §37 item 10). -/
abbrev Strategy (id : FinInfluenceDiagram) (R : Type) [CommSemiring R] :=
  (d : id.D)  $\rightarrow$  Policy id R d

namespace Policy

variable {d : id.D}

/-- A deterministic policy ' $\delta$  :  $I_D \rightarrow A_D$ ', given as a function of the whole assignment that
depends only on the information set. -/
def ofFun (f : id.Assignment  $\rightarrow$  id.states (id.action d))
  (hf :  $\forall$  x x' : id.Assignment, ( $\forall$  p  $\in$  id.info d, x p = x' p)  $\rightarrow$  f x = f x') : Policy id R d where
  kernel x a := if a = f x then 1 else 0
  localOn x x' h := by
    funext a
    dsimp only
    rw [hf x x' h]
  normalised x := by simp

@[simp] theorem ofFun_kernel (f : id.Assignment  $\rightarrow$  id.states (id.action d)) (hf)
  (x : id.Assignment) (a : id.states (id.action d)) :
  (ofFun (R := R) f hf).kernel x a = if a = f x then 1 else 0 := rfl

/-- The constant policy always choosing the action 'a' ('fix_decision', SPEC §41). -/
def const (d : id.D) (a : id.states (id.action d)) : Policy id R d where
  kernel _ b := if b = a then 1 else 0
  localOn _ _ := rfl
  normalised _ := by simp

@[simp] theorem const_kernel (d : id.D) (a : id.states (id.action d)) (x : id.Assignment)
  (b : id.states (id.action d)) : (const (R := R) d a).kernel x b = if b = a then 1 else 0 := rfl

/-- The constant policy reads no variable at all: it is local for every information set. -/
theorem const_localOn (d : id.D) (a : id.states (id.action d)) (P : Finset id.V) :
  LocalOn P (const (R := R) d a).kernel := fun _ _ => rfl

/-- In particular it is local for the empty information set. -/
theorem const_localOn_empty (d : id.D) (a : id.states (id.action d)) :
  LocalOn  $\emptyset$  (const (R := R) d a).kernel :=
  const_localOn d a  $\emptyset$ 

end Policy

namespace Strategy

/-- 'fix_decision': replace the policy of 'd' by the constant policy at 'a' (SPEC §41). -/

```

```

def fix (σ : Strategy id R) (d : id.D) (a : id.states (id.action d)) : Strategy id R :=
  Function.update σ d (Policy.const d a)

@[simp] theorem fix_self (σ : Strategy id R) (d : id.D) (a : id.states (id.action d)) :
  σ.fix d a d = Policy.const d a := by
  simp [fix]

theorem fix_of_ne (σ : Strategy id R) {d d' : id.D} (h : d' ≠ d) (a : id.states (id.action d)) :
  σ.fix d a d' = σ d' := by
  simp [fix, Function.update_of_ne h]

end Strategy

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.Instantiate

```

import InfluenceDiagramsProofs.Finite.InfluenceDiagram
import BayesianNetworksProofs.Finite.Evaluation
import Mathlib.Data.Fintype.Sum
import Mathlib.Data.Fintype.BigOperators

```

Proposition 5 (policy instantiation) for the concrete finite model (SPEC §28, §61).

`instantiate id` is the Bayesian-network shape obtained by turning every decision into a mechanism: mechanisms $M \oplus D$, targets `Sum.elim target action`, parents `Sum.elim parents info` (the `PolicyRef` mechanisms that Julia's `instantiate(id, σ)` adds before Δ -migrating into a fresh BayesNet). The strategy σ supplies the kernels of the new mechanisms: `strategyKernel κ σ = Sum.elim κ (fun d => (σ d).kernel)`.

- `closed_instantiate` — the instantiated network is closed iff the influence diagram is (`Sum.elim target action` bijective);
- `local_instantiate`, `normalised_instantiate` — the instantiated family is local and normalised when the chance kernels are, because policies are local on their information set and normalised by definition;
- `IDOrder.toTopoOrder` — an order of the influence diagram is a topological order of the instantiated network;
- hence `sum_joint_instantiate_eq_one` (Proposition 1 of the BN project applied to the instantiated network): $\sum x, \text{joint}(\text{strategyKernel } \kappa \sigma) x = 1$, the law P_σ of SPEC §31;
- `joint_instantiate` — $P_\sigma(x) = (\prod m, \kappa_m) * (\prod d, \delta_d)$, SPEC §31's factorisation.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

namespace FinInfluenceDiagram

/-- The Bayesian-network shape with one mechanism per chance variable *and* per decision.
Reducible so that '(id.instantiate).Assignment' is seen to be 'id.Assignment'. -/
@[reducible] def instantiate (id : FinInfluenceDiagram) : FinBayesNet where
  V := id.V
  M := id.M ⊕ id.D
  states := id.states
  target := Sum.elim id.target id.action
  parents := Sum.elim id.parents id.info

variable {id : FinInfluenceDiagram} {R : Type}

@[simp] theorem instantiate_target_inl (m : id.M) : id.instantiate.target (.inl m) = id.target m :=
  rfl

@[simp] theorem instantiate_target_inr (d : id.D) : id.instantiate.target (.inr d) = id.action d :=

```

```

rfl

@[simp] theorem instantiate_parents_inl (m : id.M) :
  id.instantiate.parents (.inl m) = id.parents m := rfl

@[simp] theorem instantiate_parents_inr (d : id.D) :
  id.instantiate.parents (.inr d) = id.info d := rfl

/-- **Proposition 5 (shape).** The instantiated network is closed exactly when every variable of
the influence diagram has one generator. -/
theorem closed_instantiate_iff : id.instantiate.Closed  $\leftrightarrow$  id.Closed := Iff.rfl

theorem closed_instantiate (h : id.Closed) : id.instantiate.Closed := h

/-- **Proposition 5 (order).** An order of the influence diagram is a topological order of the
instantiated network: parents precede mechanism targets and information precedes actions. -/
def IDOrder.toTopoOrder (ord : id.IDOrder) : id.instantiate.TopoOrder where
  order := ord.order
  nodup := ord.nodup
  complete := ord.complete
  parents_before := (ord.parents_before.and ord.info_before_action).imp fun {a b} h  $\Rightarrow$  by
    rintro (m | d) hm
    · exact h.1 m hm
    · exact h.2 d hm
  no_self := fun
    | .inl m  $\Rightarrow$  ord.no_self m
    | .inr d  $\Rightarrow$  ord.no_self_info d

variable [CommSemiring R]

/-- The kernels of the instantiated network: chance kernels for the chance mechanisms, policy
kernels for the decisions. -/
def strategyKernel ( $\kappa$  : id.Kernel R) ( $\sigma$  : Strategy id R) : id.instantiate.Kernel R := fun
  | .inl m  $\Rightarrow$   $\kappa$  m
  | .inr d  $\Rightarrow$  ( $\sigma$  d).kernel

@[simp] theorem strategyKernel_inl ( $\kappa$  : id.Kernel R) ( $\sigma$  : Strategy id R) (m : id.M) :
  strategyKernel  $\kappa$   $\sigma$  (.inl m) =  $\kappa$  m := rfl

@[simp] theorem strategyKernel_inr ( $\kappa$  : id.Kernel R) ( $\sigma$  : Strategy id R) (d : id.D) :
  strategyKernel  $\kappa$   $\sigma$  (.inr d) = ( $\sigma$  d).kernel := rfl

/-- **Proposition 5 (locality).** Local chance kernels and policies (local on their information
sets by definition) give a local family for the instantiated network. -/
theorem local_instantiate ( $\kappa$  : id.Kernel R) ( $\sigma$  : Strategy id R) (hloc :  $\forall$  m, Local  $\kappa$  m) :
   $\forall$  m, Local (strategyKernel  $\kappa$   $\sigma$ ) m
  | .inl m  $\Rightarrow$  hloc m
  | .inr d  $\Rightarrow$  ( $\sigma$  d).localOn

/-- **Proposition 5 (normalisation).** Normalised chance kernels and policies give a normalised
family for the instantiated network. -/
theorem normalised_instantiate ( $\kappa$  : id.Kernel R) ( $\sigma$  : Strategy id R)
  (hnorm :  $\forall$  m, Normalised  $\kappa$  m) :  $\forall$  m, Normalised (strategyKernel  $\kappa$   $\sigma$ ) m
  | .inl m  $\Rightarrow$  hnorm m
  | .inr d  $\Rightarrow$  ( $\sigma$  d).normalised

/-- **Proposition 5.** For a closed, acyclic influence diagram with local normalised chance
kernels, every complete strategy yields a Bayesian network whose joint is a probability
distribution – the law ' $P_\sigma$ ' of SPEC §31. -/
theorem sum_joint_instantiate_eq_one ( $\kappa$  : id.Kernel R) ( $\sigma$  : Strategy id R) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc :  $\forall$  m, Local  $\kappa$  m) (hnorm :  $\forall$  m, Normalised  $\kappa$  m) :
   $\sum$  x, joint (strategyKernel  $\kappa$   $\sigma$ ) x = 1 :=
  sum_joint_eq_one (strategyKernel  $\kappa$   $\sigma$ ) (closed_instantiate hclosed) ord.toTopoOrder
  (local_instantiate  $\kappa$   $\sigma$  hloc) (normalised_instantiate  $\kappa$   $\sigma$  hnorm)

/-- The joint of the instantiated network factorises as chance kernels times policy kernels

```

```

(SPEC §31, 'P_σ(x, a) = ∏ κ_X(x_X | pa_X) ∏ δ_D(a_D | i_D)'). -/
theorem joint_instantiate (κ : id.Kernel R) (σ : Strategy id R) (x : id.Assignment) :
  joint (strategyKernel κ σ) x =
    (∏ m, κ m x (x (id.target m))) * ∏ d, (σ d).kernel x (x (id.action d)) := by
  unfold joint
  exact Fintype.prod_sum_type _

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.ExpectedUtility

```

import InfluenceDiagramsProofs.Finite.Instantiate
import BayesianNetworksProofs.Finite.Intervention

```

Proposition 6 (expected utility) for the concrete finite model (SPEC §29–§31, §41, §61).

Utilities are deterministic functionals $u_j : \text{Assignment} \rightarrow R$, one per utility node, each reading only its scope (Utility.Local); the first implementation aggregates additively, $\text{totalUtility } u \ x = \sum j, u_j \ x$ (SPEC §30). The expected utility of a strategy is the expectation of the total utility under the law P_σ of the instantiated network (SPEC §31):

$\text{expectedUtility } \kappa \ \sigma \ u = \sum x, \text{ joint } (\text{strategyKernel } \kappa \ \sigma) \ x * \text{totalUtility } u \ x.$

- `expectedUtility_add`, `expectedUtility_smul` — EU is linear in the utility;
- `expectedUtility_eq_sum` — additive decomposition: $\text{EU} = \sum j, \sum x, P_\sigma(x) * u_j \ x$;
- `expectedUtility_const` — a constant total utility c has expected utility c (Proposition 5’s normalisation);
- `fix_decision`: `strategyKernel_fix_eq_intervene` — fixing decision d to a in the strategy is the hard intervention $\text{do}(\text{action } d = a)$ on the instantiated network (Proposition 4 of the BN project), so `joint_fix` gives the truncated factorisation and `expectedUtility_fix` the expected utility of a fixed decision.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

namespace FinInfluenceDiagram

variable {id : FinInfluenceDiagram} {R : Type}

```

Utilities

```

/-- A family of utility functionals, one per utility node ('utility_ref' in the ACSet). -/
abbrev Utility (id : FinInfluenceDiagram) (R : Type) := id.U → id.Assignment → R

/-- 'u_j' depends on the assignment only through the scope of 'j' (its 'UtilityInput' rows). -/
def Utility.Local (u : Utility id R) (j : id.U) : Prop :=
  ∀ x x' : id.Assignment, (∀ p ∈ id.uscope j, x p = x' p) → u j x = u j x'

variable [CommSemiring R]

/-- Additive aggregation 'U(x) = ∑ j, u_j (x_{scope j})' (SPEC §30). -/
def totalUtility (u : Utility id R) (x : id.Assignment) : R := ∑ j, u j x

/-- Expected utility of a strategy: 'EU(σ) = ∑ x, P_σ(x) U(x)' (SPEC §31; the Julia 'expected_utility(id, strategy)' reference algorithm). -/
def expectedUtility (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) : R :=
  ∑ x, joint (strategyKernel κ σ) x * totalUtility u x

/-- **Proposition 6.** 'EU(σ) = E_{P_σ}[U]': the expectation of the total utility under the

```

```

instantiated joint, written out via 'joint_instantiate'. -/
theorem expectedUtility_eq (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  expectedUtility κ σ u =
    ∑ x, ((∏ m, κ m x (x (id.target m))) * ∏ d, (σ d).kernel x (x (id.action d))) *
      ∑ j, u j x := by
  unfold expectedUtility totalUtility
  exact Finset.sum_congr rfl fun x _ => by rw [joint_instantiate]

```

Linearity

```

theorem totalUtility_add (u1 u2 : Utility id R) (x : id.Assignment) :
  totalUtility (u1 + u2) x = totalUtility u1 x + totalUtility u2 x := by
  simp [totalUtility, Finset.sum_add_distrib]

theorem totalUtility_smul (c : R) (u : Utility id R) (x : id.Assignment) :
  totalUtility (fun j y => c * u j y) x = c * totalUtility u x := by
  simp [totalUtility, Finset.mul_sum]

/-- Expected utility is additive in the utility. -/
theorem expectedUtility_add (κ : id.Kernel R) (σ : Strategy id R) (u1 u2 : Utility id R) :
  expectedUtility κ σ (u1 + u2) = expectedUtility κ σ u1 + expectedUtility κ σ u2 := by
  unfold expectedUtility
  rw [← Finset.sum_add_distrib]
  exact Finset.sum_congr rfl fun x _ => by rw [totalUtility_add, mul_add]

/-- Expected utility is homogeneous in the utility. -/
theorem expectedUtility_smul (κ : id.Kernel R) (σ : Strategy id R) (c : R) (u : Utility id R) :
  expectedUtility κ σ (fun j y => c * u j y) = c * expectedUtility κ σ u := by
  unfold expectedUtility
  rw [Finset.mul_sum]
  exact Finset.sum_congr rfl fun x _ => by rw [totalUtility_smul, mul_left_comm]

/-- Additive decomposition: the expected utility is the sum of the expected utilities of the
individual utility nodes (SPEC §30-§31). -/
theorem expectedUtility_eq_sum (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  expectedUtility κ σ u = ∑ j, ∑ x, joint (strategyKernel κ σ) x * u j x := by
  unfold expectedUtility totalUtility
  rw [Finset.sum_comm]
  exact Finset.sum_congr rfl fun x _ => Finset.mul_sum _ _ _

/-- A utility whose total is the constant 'c' has expected utility 'c' (uses Proposition 5:
the instantiated joint sums to one). -/
theorem expectedUtility_const (κ : id.Kernel R) (σ : Strategy id R) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (u : Utility id R) (c : R) (hu : ∀ x, totalUtility u x = c) :
  expectedUtility κ σ u = c := by
  unfold expectedUtility
  simp_rw [hu]
  rw [← Finset.sum_mul, sum_joint_instantiate_eq_one κ σ hclosed ord hloc hnorm, one_mul]

```

Fixing a decision

```

/-- **'fix_decision' is a hard intervention.** Replacing the policy of 'd' by the constant policy
at 'a' gives exactly the kernel family 'do(action d = a)' of the instantiated network
('intervene' of the BN project on the mechanism 'inr d'). -/
theorem strategyKernel_fix_eq_intervene (κ : id.Kernel R) (σ : Strategy id R) (d : id.D)
  (a : id.states (id.action d)) :
  strategyKernel κ (σ.fix d a) = intervene (strategyKernel κ σ) (.inr d) a := by
  funext m
  cases m with
  | inl m =>
    rw [intervene_of_ne _ Sum.inl_ne_inr]
    rfl

```

```

| inr d' =>
  by_cases h : d' = d
  · subst h
    funext x y
    show (σ.fix d' a d').kernel x y = _
    rw [Strategy.fix_self, Policy.const_kernel, intervene_self]
    rfl
  · rw [intervene_of_ne _ (fun h' => h (Sum.inr_injective h'))]
    show (σ.fix d a d').kernel = (σ d').kernel
    rw [Strategy.fix_of_ne σ h]

/-- The joint under a fixed decision: the truncated factorisation of Proposition 4. -/
theorem joint_fix (κ : id.Kernel R) (σ : Strategy id R) (d : id.D) (a : id.states (id.action d))
  (x : id.Assignment) :
  joint (strategyKernel κ (σ.fix d a)) x =
    (if x (id.action d) = a then 1 else 0) *
      (∏ m ∈ Finset.univ.erase (Sum.inr d), strategyKernel κ σ m x (x (id.instantiate.target m))) :=
  by
  rw [strategyKernel_fix_eq_intervene]
  exact joint_intervene (strategyKernel κ σ) (.inr d) a x

/-- Expected utility of a fixed decision, via the truncated factorisation. -/
theorem expectedUtility_fix (κ : id.Kernel R) (σ : Strategy id R) (d : id.D)
  (a : id.states (id.action d)) (u : Utility id R) :
  expectedUtility κ (σ.fix d a) u =
    ∑ x, (if x (id.action d) = a then 1 else 0) *
      (∏ m ∈ Finset.univ.erase (Sum.inr d), strategyKernel κ σ m x (x (id.instantiate.target m))) *
      totalUtility u x := by
  unfold expectedUtility
  exact Finset.sum_congr rfl fun x _ => by rw [joint_fix]

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.Information

```
import InfluenceDiagramsProofs.Finite.ExpectedUtility
```

Information monotonicity (SPEC §34, §35, §55.7).

An information arc $X \Rightarrow D$ enlarges the admissible policy class and nothing else: a policy that reads only `info d` also reads only a superset of it (`LocalOn` is monotone), so every strategy of an influence diagram is a strategy of the diagram with enlarged information sets, with the *same* kernels and hence the same expected utility. Consequently the optimal expected utility (supremum over strategies) cannot decrease when information is added — the requirement of SPEC §55.7 and the sign of the expected value of information (SPEC §35).

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

/-- Locality is monotone in the set of variables read. -/
theorem localOn_mono {bn : FinBayesNet} {R : Type} {P Q : Finset bn.V} (hPQ : P ⊆ Q) {v : bn.V}
  {k : bn.Assignment → bn.states v → R} (hk : LocalOn P k) : LocalOn Q k :=
  fun x x' h => hk x x' fun p hp => h p (hPQ hp)

namespace FinInfluenceDiagram

variable {id : FinInfluenceDiagram} {R : Type} [CommSemiring R]

/-- The same influence diagram with the information sets replaced by 'info'. Reducible so that
its decisions, variables and assignments are seen to be those of 'id'. -/

```

```

@[reducible] def withInfo (id : FinInfluenceDiagram) (info' : id.D → Finset id.V) :
  FinInfluenceDiagram :=
  { id with info := info' }

@[simp] theorem withInfo_info (info' : id.D → Finset id.V) (d : id.D) :
  (id.withInfo info').info d = info' d := rfl

/-- A policy for an information set is a policy for any larger one. -/
def Policy.enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d) {d : id.D}
  (π : Policy id R d) : Policy (id.withInfo info') R d where
  kernel := π.kernel
  localOn := localOn_mono (h d) π.localOn
  normalised := π.normalised

/-- A strategy of 'id' is a strategy of 'id' with enlarged information sets. -/
def Strategy.enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (σ : Strategy id R) : Strategy (id.withInfo info') R :=
  fun d => (σ d).enlarge h

/-- Enlarging the information sets does not change the kernels, hence not the joint. -/
theorem strategyKernel_enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (κ : id.Kernel R) (σ : Strategy id R) :
  strategyKernel (id := id.withInfo info') κ (σ.enlarge h) = strategyKernel κ σ := by
  funext m
  cases m <|> rfl

/-- Enlarging the information sets does not change the expected utility of a strategy. -/
theorem expectedUtility_enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  expectedUtility (id := id.withInfo info') κ (σ.enlarge h) u = expectedUtility κ σ u := by
  unfold expectedUtility
  rw [strategyKernel_enlarge]
  rfl

/-- **Information monotonicity** (SPEC §55.7). Every strategy of an influence diagram is matched,
with equal expected utility, by a strategy of the diagram with enlarged information sets; so the
optimal expected utility cannot decrease when information is added. -/
theorem exists_strategy_enlarged_eq {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  ∃ σ' : Strategy (id.withInfo info') R,
    expectedUtility (id := id.withInfo info') κ σ' u = expectedUtility κ σ u :=
  ⟨σ.enlarge h, expectedUtility_enlarge h κ σ u⟩

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

Roadmap (contains sorry)

```

import InfluenceDiagramsProofs.Finite.ExpectedUtility
import Mathlib.Data.Real.Basic

```

Module `InfluenceDiagramsProofs.Roadmap`. Statements that are **not yet proved**. This module is deliberately *not* imported by the default target (`InfluenceDiagramsProofs.lean`) and is excluded from `Audit.lean`; build it with `lake build InfluenceDiagramsProofs.Roadmap` (or `make roadmap`). Every sorry here is listed in `README.md`.

Proposition 7 — DVE correctness (SPEC §33, §61)

Decision variable elimination returns the maximal expected utility together with a *policy table*, i.e. a deterministic strategy. Its correctness therefore rests on the fact that, in the supported regular subset, deterministic policies suffice. The single-decision shadow of that statement is below: over $\mathbb{R}$, with non-negative chance kernels, some deterministic strategy dominates every non-negative strategy. (With several decisions and imperfect recall this

can fail, which is why the Julia solver restricts itself to a regular subset and is checked against exhaustive policy enumeration, SPEC §55.6.)

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

namespace FinInfluenceDiagram

variable {id : FinInfluenceDiagram}

/-- A strategy is deterministic when every policy is of the form 'Policy.ofFun'. -/
def Strategy.Deterministic (σ : Strategy id ℝ) : Prop :=
  ∀ d, ∃ f hf, σ d = Policy.ofFun f hf

/-- A strategy is non-negative when every policy kernel is. -/
def Strategy.Nonneg (σ : Strategy id ℝ) : Prop :=
  ∀ d x a, 0 ≤ (σ d).kernel x a

/-- **Proposition 7 (single-decision shadow, unproved).** For one decision, non-negative chance kernels and any utility, some deterministic strategy is optimal among non-negative strategies: the exhaustive search over policy tables of 'optimize(id; backend=Exhaustive())' finds the optimum. -/
theorem exists_deterministic_optimal [Unique id.D] (κ : id.Kernel ℝ)
  (hκ : ∀ m x y, 0 ≤ κ m x y) (u : Utility id ℝ) :
  ∃ σ' : Strategy id ℝ, σ'.Deterministic ∧
    ∀ σ : Strategy id ℝ, σ.Nonneg → expectedUtility κ σ u ≤ expectedUtility κ σ' u := by
  sorry

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```